

Plan Large, Execute Lean

A Phase-Switched Local Agent Architecture for a Single 16 GB GPU

Jack-3.8-27B-Coder-16GB-VRAM + Ornith-1.5-9B

ABSTRACT

A 16 GB GPU is powerful enough to run serious local AI models, but not powerful enough to ignore memory limits. A large model can provide strong planning and problem solving, yet consume nearly the entire GPU. A smaller model leaves room for long context, vision, embeddings, rerankers, and other specialist models, but may not be capable of solving the same difficult task from scratch.

This paper describes a local agent architecture built around that tradeoff using Jack-3.8-27B-Coder-16GB-VRAM as the main planning and audit model and Ornith-1.5-9B as an optional execution model.

The most important part of the architecture is not simply that one model plans and another executes. It is how Jack XML acts on both sides of Jack's hidden native reasoning and then survives the model boundary. Jack can perform long, exploratory native Qwen reasoning containing dead ends, temporary assumptions, rejected designs, corrections, and competing ideas. A separately loaded Ornith cannot peer into that private reasoning. What Ornith can see is the structured state Jack deliberately emits afterward.

Jack XML affects Jack internally in two distinct ways before it ever becomes a handoff artifact. First, actual XML pattern anchors - including <why>, <how>, and especially <pitfall> - are already present in Jack's prompt while native reasoning is running. Native reasoning therefore has to consider those structured attractors and failure boundaries as it searches. In earlier Jack tests, thinking-on Qwen explicitly surfaced injected security patterns inside its own reasoning, recognized requested anti-patterns as matching the active <pitfall> boundaries, and self-corrected before output. Second, Jack also knows that a mandatory output-side XML reasoning stage culminating in <pitfall_check> must be completed before any final answer. Anthropic's 2026 J-space work provides an independent mechanistic precedent: instructions can modulate a privileged internal workspace, and representations in that workspace can causally redirect multi-step reasoning and later output [1,3].

Third, after native reasoning, Jack is forced to generate structured XML before the final answer. Those XML tokens are now the most recent context at the generation frontier. Because autoregressive generation predicts each next token from the preceding context, the final answer begins from a state that has just reasserted the operative facts, constraints, provenance, and pitfall boundaries. This is frontier rebinding [2]. Fourth, when the model is switched, Ornith can read the emitted Jack XML plus the plain-language execution contract, even though it still cannot see Jack's private native reasoning.

Across 20 independent coding tests, the Ornith whole-task condition - Ornith carrying both planning and execution responsibility - was not reliable on the full difficult tasks, while Jack planning followed by Ornith execution repeatedly worked. The tested base Qwen 3.5 9B control also failed.

Execution can also remain on Jack. For ordinary text and coding tasks that do not require additional GPU resources, Jack can switch native thinking off and execute its own resolved plan without a model reload. When execution needs vision, embeddings, reranking, another inference model, or additional context headroom, Jack can be unloaded and Ornith can take over. In the tested configuration, Ornith-1.5-9B with Q8 KV cache runs at 128k context while occupying less than 10 GB of VRAM. Jack at approximately 120k context requires essentially the entire 16 GB card, and the 16 GB Jack build fits only with its vision encoder removed.

FOUR LINKED XML EFFECTS: input-side <why>/<how>/<pitfall> anchors shape native reasoning -> mandatory future <pitfall_check> constrains what the reasoning must survive -> generated XML rebinds Jack's answer frontier -> emitted XML carries visible state to Ornith.

1. The Problem Is Bigger Than Model Size

Local AI discussions often start with one question: What is the biggest model I can fit on my GPU?

For a chat model, that question makes sense. For an autonomous agent, it is incomplete.

A useful agent may need to plan a difficult software change, inspect a repository, search documentation, create embeddings, run a reranker, inspect screenshots, use a vision model, call tools, write files, execute code, examine test failures, and then reconsider the plan.

Those activities do not all place the same demands on the system. Planning benefits from the strongest reasoning model available. Execution may benefit more from free GPU memory, a larger working context, specialist models, and lower inference cost.

On a large server, it may be possible to keep everything loaded at once. On one 16 GB GPU, it is not.

That means GPU memory is not just a deployment constraint. It becomes part of the agent's decision-making architecture. The system described here treats that limitation directly.

2. The Two Models

The planning model is:

Jack-3.8-27B-Coder-16GB-VRAM Hugging Face: <https://huggingface.co/JackAgentLead/Jack-3.8-27B-Coder-16GB-VRAM>

The optional execution model is:

Ornith-1.5-9B GGUF repository: <https://huggingface.co/ornith-ai/Ornith-1.5-9B-GGUF>

Jack is used when the task needs the strongest reasoning available within the 16 GB target. It can run with native thinking enabled during planning and audit, and native thinking disabled during direct execution.

Jack XML remains active as a structured attention and state mechanism across Jack's work. Its influence begins before any new XML output is generated: input-side <why>, <how>, and <pitfall> pattern anchors are already in the prompt that native reasoning must consider, while the separate requirement to complete a later <pitfall_check> creates a mandatory falsification checkpoint before the final answer. After native reasoning, Jack generates the output-side XML checkpoint immediately before the final answer, rebinding the operative state near the token-generation frontier. During Jack thinking-off execution, the same XML anchoring remains active even though native deliberative reasoning is disabled. If execution is handed to Ornith, the emitted XML remains visible in the conversation and becomes structured context for the downstream model.

Ornith is much smaller. That smaller size matters not just because generation can be cheaper. It changes what else can fit on the GPU.

In the tested setup, Ornith at 128k context with Q8 KV cache stays below 10 GB of VRAM. Jack, at roughly 120k context, consumes essentially the full 16 GB GPU.

There is another important difference: the 16 GB Jack build fits its target hardware only without the vision encoder. So the two models naturally create two very different GPU environments.

3. The Test Result That Changed the Architecture

At first, the obvious idea was simply: Jack plans -> Ornith executes .

But the testing showed something more useful.

Across the 20 independent tests, when Ornith was given the full difficult coding problem and had to carry the whole task from planning through execution, it was not reliable. The tested base Qwen 3.5 9B control also failed.

When Jack first solved the planning problem and Ornith received the resolved execution task, the combination repeatedly worked across the 20-test array.

Twenty independent tests were run. This paper presents one representative task in detail rather than reproducing all 20 logs; the purpose is to expose the architecture clearly enough for widespread independent contribution, replication, and stress testing.

That is an important distinction. It does not show that Ornith suddenly gained the reasoning ability of a 27B model. It shows that the work presented to Ornith had changed.

The original task might require the model to discover a data structure, reason through several invariants, resolve ambiguous interactions between requirements, decide validation order, anticipate failures, choose an algorithm, write the implementation, and audit it.

After Jack finishes planning, much of that search is gone. Ornith receives something closer to an engineering specification. That is a much smaller problem.

4. A Planner Should Not Hand Its Thought Process to the Executor

This is the most important design principle in the system. A naive planner-executor architecture might do this: Large model thinks for a long time -> Copy the entire reasoning transcript -> Small model reads it -> Small model tries to execute .

That looks reasonable. It can also be a bad idea.

Long reasoning is exploratory by nature. A planner may temporarily consider a heap, then reject it. It may consider adding a reverse map, then discover that the task prohibits one. It may misunderstand a requirement, correct itself fifty lines later, revisit a previous design, and finally settle on a modified version of the first idea.

All of that can be useful while solving the problem. None of it necessarily belongs in the executor's context.

Consider a simplified planning trace:

```
Maybe use a heap.  
No, that complicates ownership.  
Could maintain seq_to_event.  
Wait, the task forbids another persistent reverse map.  
Maybe scan seen.  
That works but seems unnecessary.  
What if pending itself is keyed by seq?  
Yes.  
Now exception safety means the conflict check must happen before writing to seen.  
Use pending keyed by seq.
```

The executor did not need the discarded branches of that search. It needed the conclusions that survived: the selected state representation, conflict rules, mutation order, failure boundaries, and audit conditions. Passing the raw transcript would force a 9B executor to sort through ideas that Jack had already rejected. That is backwards.

5. The Core Boundary: Ornith Sees Jack XML, Not Jack's Raw Reasoning

In this architecture, Jack's private native reasoning, Jack's XML instructions, and Jack's emitted XML state are three different things. The distinction matters.

Before native reasoning starts, Jack has already received two different forms of XML control. The first is actual input-side XML pattern content: <why> supplies rationale, <how> supplies the constructive path, and <pitfall> supplies an explicit negative boundary that native reasoning has to consider while it searches. The second is the instruction that a later output-side <pitfall_check> must be completed before the final answer. The newly generated checkpoint tokens do not yet exist, but the pattern anchors are already active context and the future falsification gate is already mandatory.

During planning, Jack may spend minutes inside its native Qwen reasoning process exploring alternatives, correcting itself, testing constraints, and discarding dead ends. A separately loaded Ornith cannot peer into that hidden reasoning. After the reasoning stage, however, Jack emits the XML checkpoint into the visible assistant turn. That XML carries the selected design, constraints, invariants, variables, failure conditions, implementation order, provenance, and audit targets forward.

The distinction is foundational. Ornith does not run the Jack XML protocol as its own reasoning system, but it can read and use the XML artifact Jack produced. The plain-language technical execution contract follows the XML and gives Ornith direct implementation instructions. The downstream model therefore receives both the structured operative state and the clean contract while Jack's raw native reasoning remains private.

JACK CONTROL CONTEXT

Input-side <why> / <how> / <pitfall> patterns + mandatory future <pitfall_check>
[visible to Jack; shapes the reasoning environment]

JACK PRIVATE NATIVE REASONING

Exploration, alternatives, corrections, discarded paths
[hidden from Ornith]

JACK XML - VISIBLE CHECKPOINT

Operative state, provenance, anchors, constraints, <pitfall_check>, failure boundaries, audit targets
[frontier rebinding for Jack; visible to Ornith after model switch]

EXECUTION CONTRACT - VISIBLE OUTPUT

Plain-language implementation instructions conditioned on the XML checkpoint

ORNITH-1.5-9B

Reads Jack XML + execution contract; uses its own native reasoning to execute

This is more than context trimming. Jack XML first supplies active input-side <why>/<how>/<pitfall> anchors that can shape Jack's hidden reasoning, while the mandatory future <pitfall_check> adds a separate prospective falsification constraint. The generated XML then becomes a visible frontier-rebinding checkpoint for Jack's own final answer and, if the model is switched, a cross-model state bridge that Ornith can read.

6. Jack XML Works Through Four Linked Effects

Jack XML should not be confused with a formatting style that Ornith must adopt. It is a control structure with four linked effects across three locations in the workflow: input-side workspace anchoring and prospective checkpoint pressure inside Jack's reasoning environment, frontier rebinding at Jack's final-output boundary, and visible state transfer across the model boundary.

Effect 1 - input-side workspace anchoring. Before private reasoning begins, Jack already sees actual structured XML pattern cards. `<why>` establishes the rationale, `<how>` establishes the constructive direction, and `<pitfall>` establishes a concrete failure boundary. Native reasoning therefore has to process the task in the presence of those anchors rather than reasoning in an XML-free space. In the earlier Jack thinking-on / XML-on security experiment, Qwen's own reasoning explicitly identified the injected security patterns, recognized that the user's requested design matched the active `<pitfall>` conditions, and self-corrected before producing code [3].

Effect 2 - prospective checkpoint pressure. Separately from the `<pitfall>` patterns that are already in context, Jack knows that it is not allowed to produce the final answer until the later XML reasoning stage has reconstructed the operative state and completed `<pitfall_check>`. The private reasoning process therefore occurs under a known future falsification requirement: the reasoning must eventually survive an explicit check of evidence, assumptions, anchors, counterarguments, and failure modes before the answer is permitted to begin.

Anthropic's J-space results provide a useful mechanistic precedent for these inward effects. Anthropic found that a model's privileged workspace can be modulated by instruction, that intermediate concepts in that workspace causally mediate multi-step reasoning, and that altering a workspace representation can redirect downstream answers [1]. Those experiments were performed on Claude, not on this Qwen-derived Jack model, so they are not a direct measurement of Jack. They do, however, support the general mechanism that structured instructions about what the model should hold, pursue, inspect, or avoid can change the internal reasoning state before any visible answer is written.

Effect 3 - outside native reasoning: frontier rebinding. After hidden reasoning finishes, Jack must generate the XML checkpoint before the final answer. The XML is now literal, recent token context. `<workspace_state>`, `<grounded_source>`, `<anchor_fact>`, `<pitfall_check>`, and any deterministic verification state are placed immediately before action. Every token of the final answer is autoregressively conditioned on that refreshed context from the first answer token onward. Jack's earlier Rolling Cognitive State Maintenance paper calls this frontier rebinding: do not merely leave an important variable somewhere in memory; rebind it near action [2].

Effect 4 - across the model boundary: visible state transfer. If execution switches to Ornith, the emitted XML remains in the conversation. Ornith can read those anchors plus the execution contract even though it cannot inspect Jack's hidden reasoning. The same XML architecture has therefore already influenced Jack through input-side pattern anchoring and prospective checkpoint pressure, then frontier-rebound Jack's own answer, before the emitted state becomes the visible handoff to the downstream executor.

The full sequence is: input-side `<why>/<how>/<pitfall>` anchors already present in Jack's prompt + mandatory future `<pitfall_check>` -> private native reasoning shaped under those controls -> mandatory visible XML reasoning checkpoint -> frontier-rebound final plan / execution contract -> model switch -> Ornith reads Jack XML + contract and executes with its own native reasoning.

INSIDE NATIVE REASONING Effect 1: <code><why>/<how>/<pitfall></code> anchors are already in context and can steer the private search. Effect 2: mandatory future <code><pitfall_check></code> gives that search a checkpoint it must later survive.	AT THE OUTPUT FRONTIER Effect 3: generated Jack XML is placed immediately before the answer, refreshing the state that conditions the next-token distribution.	ACROSS THE MODEL BOUNDARY Effect 4: the emitted XML remains visible. Ornith reads the structured state + execution contract, not Jack's private reasoning trace.
--	--	--

7. The Executor Gets Structured Reasoning State and a Blueprint, Not a Diary

The handoff to Ornith contains two useful layers: the visible Jack XML reasoning state and the plain-language engineering contract that follows it. It does not contain Jack's raw native reasoning diary.

For a coding task, Ornith may receive a fixed data representation, the exact order of validation, negative constraints, required mutation ordering, defined error behavior, implementation steps, and acceptance tests. It may receive instructions such as:

```
seen is keyed by event_id.  
pending is keyed by seq.  
Do not create an additional persistent ownership map.  
Check exact duplicate before any state mutation.  
Check event-ID conflicts before accepting the event.  
Only mutate persistent state after every failing validation for the current event has passed.  
Apply the current sequence once.  
Then drain pending only while next_seq exists.  
Never sort the complete input.  
Audit duplicate delivery, gaps, conflicts, and failed-event state preservation before returning.
```

There is very little architectural ambiguity left. Ornith still has to write correct code and use its own native reasoning. But it can read the XML anchors that preserve what Jack resolved and the contract that tells it what to build. It is no longer being asked to infer which of several hidden approaches survived Jack's planning stage.

8. Cognitive Pollution

A useful term for the problem avoided here is cognitive pollution.

Suppose Jack briefly considers `seq_to_event` inside native reasoning and later determines that adding such a persistent map violates the task. That rejected thought remains private. The visible Jack XML should carry the surviving constraint - for example, never add an auxiliary persistent ownership map - rather than exposing the discarded proposal as active state. This reduces the chance that a downstream executor revives a dead end.

The same thing happens with an abandoned heap, a wrong assumption about the input, an earlier validation order, an obsolete interface, or a temporary interpretation of an edge case.

A raw reasoning trace mixes things considered, rejected, corrected, and accepted. Jack XML exposes the current operative state instead. Ornith can see that state directly without being forced to reconstruct it from the hidden search history.

9. Context Is Not Free Just Because It Is 128k

Ornith can run at 128k context in the tested configuration. That does not mean the system should waste that context.

During a real coding session, the execution model may need room for source files, retrieved documentation, tool results, compiler errors, tests, execution history, the original task, Jack's contract, and later revisions.

Every thousand tokens of unnecessary planning debate would occupy context that could instead hold active execution information. The XML handoff avoids that problem because Ornith sees compact structured reasoning anchors rather than Jack's full hidden chain of thought. The purpose is not merely

to make the prompt shorter; it is to improve the signal-to-noise ratio of the executor's context while preserving the reasoning state that matters.

The goal is not: Give the executor as much information as possible. It is: Give the executor as much useful current information as possible.

10. Why This Matters for Agent Systems in General

This principle is larger than Jack and Ornith. Many agent architectures use a planner, worker, critic, coder, or specialist model. A common assumption is that more shared context is always better. It is not necessarily better.

If a planner copies a raw search transcript into the worker's context, the worker inherits the planner's discarded hypotheses as well as its conclusions. Jack takes a different approach: native reasoning remains private, while structured operative state is emitted in XML and becomes visible to the next role. That can prevent several problems:

The worker may revive rejected approaches.
It may confuse hypotheses with requirements.
It may waste reasoning on decisions the planner already settled.
It may anchor on a sentence that was later corrected.
It may spend valuable context interpreting another model's reasoning instead of acting.

Jack: input-side <why>/<how>/<pitfall> anchors + future <pitfall_check> -> private search -> visible XML checkpoint -> action
Cross-model: visible XML checkpoint -> execution contract -> Ornith

The broader lesson is not merely that multi-model agents need a visible state bridge. In Jack, the same XML architecture influences the planner before the handoff exists: input-side <why>/<how>/<pitfall> patterns are already present during private reasoning, the separate future <pitfall_check> constrains what that reasoning must eventually survive, the emitted XML then rebinds the answer frontier, and the same visible state can be consumed by the next model. A robust agent interface can therefore align search, falsification, action, and handoff with one structured representation.

11. The Ornith Whole-Task Control

The Ornith whole-task tests help explain why this matters. In this planner-and-executor control, Ornith received the original difficult problem and had to perform both roles itself: discover architecture + resolve constraints + handle edge cases + implement + audit. Across the 20 independent tests, that whole-task condition was not reliable.

In the Jack + Ornith condition, Jack first performed the difficult architectural search under the input-side XML pattern anchors and the mandatory output-side <pitfall_check> requirement. Jack then emitted the structured operative state and execution contract. Ornith entered the same conversation able to see those XML anchors plus the contract, while still seeing none of Jack's private native reasoning. Its job became closer to: consume resolved state + follow contract + implement + perform local checking. That condition repeatedly worked across the 20-test array.

This supports a simple interpretation: The original task was outside Ornith's reliable whole-task capability range, while the resolved execution problem was often inside it. That is not the same thing as saying Ornith is equivalent to Jack. It is evidence that task decomposition can move work into a smaller model's usable capability range.

12. The Base Qwen 3.5 9B Control

The tested base Qwen 3.5 9B control also failed. That is useful because it prevents an overly simple conclusion. The result is not: Any 9B model can execute after planning.

The execution model still matters. A small executor must preserve detailed constraints, follow a contract faithfully, avoid redesigning the problem unnecessarily, write correct code, and handle local implementation details.

Planning reduces the amount of intelligence required. It does not remove the need for intelligence. Ornith appears useful in this role because its specialization and behavior make it a better fit for the narrowed execution problem than the tested base 9B control.

13. Jack Can Also Execute Its Own Plan

Jack + Ornith is not always the best path. After Jack finishes planning, the 27B model is already loaded. If the next step is simply to write code or produce text, unloading Jack and loading Ornith may be pointless.

Jack can instead switch native thinking off. The workflow becomes:

PLAN	EXECUTE	AUDIT
Jack thinking ON + Jack XML	Jack thinking OFF + Jack XML	Jack thinking ON + Jack XML

There is no model switch between planning and execution. Thinking off does not remove Jack's learned coding ability. It removes the expensive deliberate reasoning stage because the architecture has already been resolved. For straightforward coding, this can be the simplest and fastest execution path.

14. Why Ornith Becomes More Attractive When Execution Needs More Models

The comparison changes when execution needs more GPU resources. At large context, Jack effectively occupies the entire 16 GB card. That means the execution phase has little room for anything else.

Ornith's tested memory profile is very different:

Configuration	Context	VRAM
Jack-3.8-27B-Coder	~120k	essentially full 16 GB
Ornith-1.5-9B + Q8 KV	128k	less than 10 GB

So Ornith leaves more than 6 GB of nominal VRAM headroom while also carrying a slightly larger tested context. That headroom must still absorb runtime overhead, buffers, and any additional execution components. This changes the purpose of the model switch. It is no longer: Load a smaller model because small is faster.

It becomes: Load a smaller execution model because execution now needs an entire collection of resources that cannot coexist with Jack.

15. Vision Is the Clearest Example

The 16 GB Jack configuration fits only after removal of the vision encoder. So Jack cannot provide the original vision capability while maintaining this target memory footprint.

If an agent needs repeated visual inspection, keeping Jack as the only execution resident can lead to repeated model changes. For example: Jack -> unload -> vision model -> inspect screenshot -> unload ->

Jack -> continue coding -> unload -> vision model -> inspect again . That is not attractive for a visual development workflow.

Now consider the Ornith execution phase. If Ornith remains below 10 GB, the system has more than 6 GB of nominal VRAM headroom for runtime overhead and additional execution components. Depending on the model chosen, an environment such as Ornith + vision model may remain resident through the execution loop. Now the agent can repeatedly inspect, edit, run, and inspect again without rebuilding the entire GPU residency after each step. That is a real system-level benefit.

16. Embeddings, Reranking, and Retrieval

The same principle applies to repository search. A long coding task may involve far more source code than should be placed in context at once.

An embedder can identify relevant files. A reranker can improve the retrieved set. The execution environment might therefore need Ornith + 128k KV + embedder + reranker rather than one giant language model consuming almost the entire GPU.

Some of these jobs can be moved to CPU. That may be perfectly reasonable. The goal is not to force every model onto the GPU. The point is that Ornith gives the scheduler more options. Jack leaves very little GPU flexibility at its long-context configuration. Ornith leaves substantial room.

17. The Agent Chooses Between Two Execution Modes

The best architecture is therefore not permanently Jack -> Ornith or permanently Jack -> Jack . It is:

JACK THINKING-OFF	ORNITH-1.5-9B
Text / code execution	Extra VRAM needed
No major co-resident model needed	Vision / embeddings / reranking
No model swap required	Specialists / larger execution headroom

This gives the system a practical rule: If Jack already fits the entire execution workload, keep Jack. If execution needs the GPU for more than Jack, switch to the smaller execution environment.

18. Planning, Execution, Verification, Audit

The full cycle is better written as: PLAN -> EXECUTE -> VERIFY -> AUDIT / REPLAN.

Jack owns planning. Either Jack thinking-off or Ornith owns execution. Deterministic software should perform verification whenever possible. Jack returns for audit or replanning when the workflow requires higher-order review or when verification exposes unresolved architectural uncertainty.

PLAN	EXECUTE	VERIFY	AUDIT / REPLAN
Jack thinking ON Jack XML active	Jack no-think + XML or Ornith reading Jack XML	Deterministic tests / checks	Jack thinking ON when required

This keeps the expensive model focused on the stages where it is actually valuable.

19. Verification Should Not Depend on Model Confidence

One lesson from the testing is that a model can deeply reason about its own output, describe why the code is correct, and still miss a small implementation mistake. That means the first verification step should usually be deterministic.

For code, that means real execution, tests, compilation, static checks, schema validation, or other machine-verifiable evidence. If the code passes, there may be no need to reload Jack. If it fails locally, Ornith may be able to repair it. If the failure reveals that the architecture itself was wrong or incomplete, the system returns to Jack.

The model's confidence is not the gate. The evidence is.

20. The GPU Has Two Different Jobs

The architecture can be understood as two GPU modes:

COGNITIVE MODE	EXECUTION MODE
Jack 27B native thinking ON Jack XML long context	Ornith 9B 128k context Q8 KV <10 GB
planning - architecture - diagnosis - difficult reasoning - audit	>6 GB nominal headroom for vision - embeddings - reranking - specialist models - buffers
Objective: maximize reasoning capability.	Objective: maximize execution capability.

The same physical GPU is therefore used in two very different ways during one task.

21. The Bigger Agent-System Lesson

The most important idea may ultimately have little to do with 9B versus 27B. It is about how agents communicate between cognitive stages.

A planner should be free to explore privately, but the planner's private search is not isolated from its own control structure. In Jack, actual input-side <why>/<how>/<pitfall> patterns are already part of the prompt shaping native reasoning, while the separate mandatory future <pitfall_check> requires that search to survive a later falsification checkpoint. After reasoning, the generated XML reasserts the operative state immediately before action. If another model takes over, that same emitted XML becomes the visible state bridge. One structure therefore links internal reasoning discipline, falsification, output steering, and cross-model transfer.

That sounds obvious. In practice, many systems blur the two. They carry long histories forward because the history might contain something useful. That can work, but it places a growing burden on every downstream model to distinguish current truth from historical speculation.

Structured state gives the system another option. Jack's private reasoning remains private, but it is conducted with explicit XML pattern anchors already in context and a mandatory <pitfall_check> still ahead; the resulting XML is then emitted at the frontier and becomes visible to the downstream executor. This becomes increasingly important as agent workflows get longer, involve several models, or use smaller specialists that cannot afford to rediscover a larger planner's decisions.

22. Search Is Not State

There is a simple conceptual distinction underneath all of this. Reasoning is a search process. Execution requires a state.

Search naturally contains branches. State should contain what is currently believed, required, selected, or unresolved. Those should not be treated as the same thing.

Jack's native reasoning performs the private search with input-side `<why>/<how>/<pitfall>` anchors already present and a mandatory future `<pitfall_check>` still ahead. The generated Jack XML then reconstructs and rebinds the operative state at the action frontier. The execution contract turns that state into direct instructions. If Ornith is selected, it reads both and acts on them. The architecture is therefore not simply Search -> State -> Action; it is constrained Search -> explicit State Checkpoint -> Action -> reusable State Handoff.

Clean division: Search -> State -> Action
rather than: Search -> Search Transcript -> Another Search -> Action

For a multi-model agent, that difference can be enormous.

23. The Experimental Interpretation

The 20-test independent array supports a specific, limited conclusion: the Ornith whole-task condition did not reliably solve the tested difficult tasks. The tested base Qwen 3.5 9B control also failed. Jack + Ornith repeatedly succeeded when Jack first performed the difficult planning and Ornith received the resolved execution problem. This paper gives one representative task in detail; the other 19 tests are not reproduced here, and the architecture is intentionally presented for broader independent contribution, replication, and stress testing.

This does not prove that every 9B model requires a 27B planner. It does not prove Jack + Ornith is always faster than Jack + Jack thinking-off. It does not prove every planned task will be implemented correctly by Ornith.

What the 20-test array shows is an important capability split in the tested system: a model can be below the reliable threshold for solving the entire problem while remaining above the threshold for executing a properly resolved version of that problem. The Jack XML boundary is central because input-side `<why>/<how>/<pitfall>` anchors shape Jack's planning environment, the mandatory future `<pitfall_check>` adds a falsification gate, the generated XML rebinds Jack's output frontier, and the emitted state gives Ornith visible access to operative reasoning state that would otherwise remain trapped behind Jack's private reasoning boundary. These are behavioral systems experiments rather than direct Jacobian-lens measurements of Qwen; Anthropic's J-space findings are used here as mechanistic context, not as a claim that Jack's internal activations were directly instrumented [1].

24. How the System Should Be Benchmarked

A model-only benchmark is not enough. The important quantity is the whole workflow.

For Jack thinking-off execution, the cost includes planning, execution, verification, and any audit. For Ornith execution, it additionally includes unloading Jack, loading Ornith, auxiliary model work, and possibly reloading Jack.

But a multimodal Jack-only workflow may itself require repeated unloading and loading to make room for vision or other components. The meaningful comparison is therefore total wall-clock task completion at comparable correctness. Peak VRAM matters. Number of residency changes matters. Repair count matters. First-pass success matters. Context capacity matters. The amount of useful GPU capability available during execution matters. Tokens per second alone does not capture any of that.

25. Why the 16 GB Target Makes This Interesting

There would be less reason for this architecture if unlimited memory were available. A 48 GB GPU might hold a large model, vision, embeddings, and a huge KV cache at once. A 16 GB card forces a decision. That constraint creates the opportunity.

Jack can consume almost the whole GPU while the task is cognitively difficult. Then the system can deliberately give up some model capacity once those hard decisions are settled. The freed memory becomes useful execution infrastructure.

This is not merely model swapping. It is phase-dependent resource allocation.

26. Final Architecture

The complete design can be summarized like this:

TASK

Original task enters Jack

JACK CONTROL CONTEXT + PRIVATE THINKING

Input-side <why>/<how>/<pitfall> patterns are already active and a future <pitfall_check> is mandatory; native Qwen reasoning stays hidden

VISIBLE JACK XML CHECKPOINT

Reconstructs operative state and frontier-rebinds the final output; remains visible after model switch

EXECUTION CHOICE

Jack thinking-off + XML for simple execution | Ornith 9B reads Jack XML + contract for resource-rich execution

DETERMINISTIC CHECKS -> AUDIT / REPLAN

Pass -> done or scheduled audit; fail / uncertainty -> repair or reload Jack for deeper audit / replanning

27. Conclusion

The main lesson from this work is not that a 27B model is better than a 9B model. The useful lesson is that planning, execution, and audit are different computational jobs.

Jack-3.8-27B-Coder-16GB-VRAM is used when the problem still contains substantial uncertainty. It performs the difficult search. That search can be messy. It can contain wrong turns. It can contain ideas that should never reach the final implementation. That is acceptable because the raw search process is not the handoff.

Jack XML operates on both sides of that hidden reasoning stage. Before native reasoning begins, actual input-side XML pattern anchors are already in context: <why> carries rationale, <how> carries the constructive direction, and <pitfall> carries explicit failure boundaries that native reasoning has to consider. Separately, Jack knows that the later XML reasoning stage must complete <pitfall_check> before any final answer is allowed. After native reasoning, Jack generates that output-side XML checkpoint immediately before the final answer, rebinding the selected architecture, constraints, provenance, invariants, implementation order, and audit requirements near the generation frontier.

Those newly generated XML tokens then condition Jack's final answer from its first token onward. If the model is switched, the same XML remains visible to Ornith. Ornith therefore receives both the structured reasoning state and the plain-language technical contract; what it never receives is Jack's private native Qwen chain of thought.

This is the crucial four-effect mechanism: input-side `<why>/<how>/<pitfall>` anchors influence the hidden search; the mandatory future `<pitfall_check>` gives that search a prospective falsification target; the generated XML steers Jack's own answer at the token frontier; and the emitted XML exposes the operative state across the model boundary. The architecture does not ask Ornith to reconstruct what happened inside Jack. It makes the state that matters visible before Ornith begins.

That is particularly valuable when the downstream model is smaller. It prevents the executor from spending part of its limited reasoning capacity sorting through another model's abandoned ideas and historical uncertainty.

The 20 independent tests fit this interpretation. The Ornith whole-task condition was not reliable on the full difficult tasks. The tested base Qwen 3.5 9B control also failed. Jack + Ornith repeatedly worked when the difficult planning problem was resolved first.

The execution model can then be chosen according to the hardware needs of the task. For straightforward text and coding work, Jack stays loaded and executes with native thinking disabled. For execution that needs vision, embeddings, reranking, additional models, or greater VRAM headroom, Jack can be unloaded and Ornith becomes the execution resident.

In the tested configuration, Ornith provides 128k context with Q8 KV cache while remaining below 10 GB VRAM. Jack at roughly 120k consumes essentially the entire 16 GB card and fits the target only without its vision encoder.

So the system uses one GPU in two ways: When uncertainty is high, spend the GPU on cognition. When the problem has been resolved, spend the GPU on execution. And between those two stages: transfer the resolved state, not the reasoning mess.

For multi-model agent systems, that last principle may be the most important result of all.

Models

Jack-3.8-27B-Coder-16GB-VRAM

<https://huggingface.co/JackAgentLead/Jack-3.8-27B-Coder-16GB-VRAM>

Ornith-1.5-9B

<https://huggingface.co/ornith-ai/Ornith-1.5-9B-GGUF>

Additional tested control: Base Qwen 3.5 9B.

Related Work and Mechanistic Context

[1] Anthropic. A global workspace in language models. July 6, 2026. The J-space work reports that privileged internal representations can be modulated by instruction, causally mediate multi-step reasoning, and redirect downstream outputs when intervened upon.

[2] Jonathan Michael Langford. Rolling Cognitive State Maintenance: Why Structured Carry-Forward May Matter More Than Raw Context Length. Introduces rolling operative state and frontier rebinding: place structured state immediately before action so controlling variables remain behaviorally salient.

[3] Jonathan Michael Langford. Engineering the J-Space: Beyond Chain of Thought. Reports contrastive Qwen3.5-9B-MTP tests in which XML safety patterns, especially `<pitfall>`, altered both thinking-on reasoning behavior and final execution behavior.

[4] Jonathan Michael Langford. Engineering J-Space for Long-Context Recall. Develops the use of structured `<why>`, `<how>`, and `<pitfall>` anchors as a long-context workspace interface and reports long-horizon retrieval / alignment experiments.